

LABORATORIO DI PROGRAMMAZIONE
CORSO DI LAUREA IN SICUREZZE DEI SISTEMI E DELLE RETI
INFORMATICHE
UNIVERSITÀ DEGLI STUDI DI MILANO
2025–2026

INDICE

Parte 1. Allocazione dinamica della memoria	2
Esercizio 1	2
<i>Ordinamento BubbleSort con allocazione dinamica</i>	2
Tempo: 25 min.	2
Esercizio 2	2
<i>Ordinamento BubbleSort con allocazione dinamica, menu, e funzioni</i>	2
Tempo: 35 min.	2
Esercizio 3	2
<i>Ordinamento BubbleSort con riallocazione dinamica</i>	2
Tempo: 30 min.	3
Esercizio 4	3
<i>Unire e dividere due array con allocazione dinamica</i>	3
Tempo: 25 min.	3
Esercizio 5	3
<i>Leggere una frase e creare una struttura dati per contenerla</i>	3
Tempo: 10 min.	3
Parte 2. File e argc/argv	3
Esercizio 6	3
<i>Scrivere il contenuto di un array su file</i>	3
Tempo: 20 min.	4
Esercizio 7	4
<i>Leggere il contenuto di un array da file</i>	4
Tempo: 20 min.	4
Esercizio 8	4
<i>Leggere il contenuto di un array da file (versione 2)</i>	4
Tempo: 20 min.	4

Parte 1. Allocazione dinamica della memoria

In questa prima parte della lezione, per allocare dinamicamente la memoria userete le funzioni `malloc`, `calloc`, `realloc` e `free` dichiarate nel file di intestazione `stdlib.h`.

ESERCIZIO 1

Ordinamento BubbleSort con allocazione dinamica.

Tempo: 25 min.

Si scriva un programma che legga dalla console una successione di valori `double`, e li memorizzi in una zona della memoria di dimensione appropriata. Chiedete all'utente per prima cosa quanti valori `double` intenda inserire. Usate poi questa informazione per eseguire un'appropriata chiamata alla funzione `malloc`. Dopo aver letto e memorizzato i valori, il programma li ordina applicando l'algoritmo `BubbleSort` — si veda l'Esercizio 11 della Lezione 5 — visualizza l'elenco ordinato, libera la memoria allocata dinamicamente e termina.

ESERCIZIO 2

Ordinamento BubbleSort con allocazione dinamica, menu, e funzioni.

Tempo: 35 min.

Si scriva un programma che visualizzi un menu come segue:

1. Inserisci elenco di `double`.
2. Ordina elenco.
3. Visualizza elenco.
4. Termina.

Se l'utente sceglie 4 il programma termina. Se l'utente sceglie 2 o 3 prima di aver inserito un elenco di `double`, il programma informa l'utente della necessità di inserire dei dati e torna al menu. Se l'utente sceglie 1 il programma permette all'utente di inserire un elenco di valori di tipo `double`, e li memorizza usando le funzioni di `stdlib.h` per l'allocazione dinamica della memoria. Chiedete preliminarmente all'utente quanti valori intende inserire, come nell'Esercizio 1. Stabilite un comportamento sensato del programma nel caso in cui l'utente scelga 1 e inserisca, o tenti di inserire, zero valori. L'opzione 3 permette di visualizzare l'elenco corrente. L'opzione 2 ordina l'elenco corrente applicando l'algoritmo `BubbleSort`. Se l'utente sceglie 1 dopo aver già inserito un elenco, il programma scarta il vecchio elenco (deallocando la memoria con `free`) e permette all'utente di inserirne uno nuovo, allocando la memoria necessaria di conseguenza. Strutturate il programma in funzioni appropriate.

ESERCIZIO 3

Ordinamento BubbleSort con riallocazione dinamica.

Tempo: 30 min.

Si modifichi il programma scritto per risolvere l'Esercizio 2 di modo che il menu diventi:

1. Inserisci elenco di `double`.
2. Ordina elenco.
3. Visualizza elenco.
4. Aggiungi elementi.
5. Termina.

L'opzione 4 permette all'utente di aggiungere valori `double` in coda all'elenco di valori corrente. Chiedete preliminarmente all'utente quanti valori intende aggiungere all'elenco corrente. Usate `realloc` per ridimensionare la memoria allocata dinamicamente. Se l'utente sceglie 4 prima di aver inserito un elenco di `double` tramite 1, il programma informa l'utente della necessità di inserire dei dati e torna al menu. Per il resto il funzionamento del programma è come nell'Esercizio 2.

ESERCIZIO 4

Unire e dividere due array con allocazione dinamica.

Tempo: 25 min.

Si scriva un programma che legga dalla console due successioni di valori `int`, inseriti dall'utente, che verranno memorizzati all'interno di due aree di memoria, allocate staticamente, di dimensione `N = 100`. Chiedete all'utente per prima cosa quanti valori `int` intenda inserire per entrambi gli array. Implementate una funzione `merge` che, ricevuti tali array e il numero di elementi contenuti in essi, allochi dinamicamente un nuovo array che contenga esattamente tutti e soli i valori dei due array, e lo restituisca. Se il primo array contiene 3 valori, e il secondo ne conviene 5, l'array creato ne dovrà contenere esattamente 8.

ESERCIZIO 5

Leggere una frase e creare una struttura dati per contenerla.

Tempo: 10 min.

Si scriva un programma che chieda all'utente quante parole voglia inserire. Tali parole (senza spazi o caratteri speciali) dovranno essere inserite in un array di puntatori a stringhe di dimensione `N = 20`. Per ciascuna di tale parole, inserite dall'utente, si allochi dinamicamente una nuova stringa della dimensione esatta della parola inserita. Stampare a schermo la frase ottenuta. Completate per farlo il codice di Fig. 1.

Parte 2. File e `argc/argv`

ESERCIZIO 6

Scrivere il contenuto di un array su file.

Tempo: 20 min.

Si scriva un programma che legga dalla console una successione di valori `double`, e li memorizzi in una zona della memoria di dimensione appropriata. Chiedete all'utente per prima cosa quanti valori `double` intenda inserire. Usate poi questa informazione per eseguire un'appropriata chiamata alla funzione `malloc`. Dopo aver letto e memorizzato i valori, il programma li memorizza su un file specificato tramite `argv`. Il programma deve verificare che il file esista, e, in caso, crearlo. Salvate un valore per riga nel file.

ESERCIZIO 7

Leggere il contenuto di un array da file.

Tempo: 20 min.

Si scriva un programma che, dato un file specificato tramite `argv`, legga il contenuto del file riga per riga assumendo che segua lo stesso formato definito nell'Esercizio 6. La sequenza di valori dev'essere memorizzata in un array, la cui dimensione massima è nota a priori: assicuratevi che non venga sforata durante la lettura da file. Il programma visualizza quindi l'array.

ESERCIZIO 8

Leggere il contenuto di un array da file (versione 2).

Tempo: 20 min.

Modificate il programma dell'esercizio precedente allocando dinamicamente la memoria per l'array. Una volta allocato dinamicamente l'array, il programma legge il contenuto del file riga per riga e lo assegna all'array. Il programma visualizza quindi l'array.

DIPARTIMENTO DI INFORMATICA, UNIVERSITÀ DEGLI STUDI DI MILANO,

Per farlo, è necessario conoscere *prima* il numero di righe del file: usate le funzioni di lettura che conoscete.

Le operazioni di I/O sono costose, perché la loro velocità di lettura/scrittura è molto inferiore rispetto alla RAM. In un programma reale, occorre bilanciare il consumo della memoria con il tempo di esecuzione.

La funzione `void rewind(FILE * file)` posiziona `file` all'inizio per le operazioni successive.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define N 20
6  #define MAX_WORD_LENGTH 50
7
8  int main() {
9      // Chiede all'utente quanti parole vuole inserire
10     int numParole, i;
11     // Alloca staticamente un array di puntatori a stringhe di dimensione N
12     char *parole[N];
13     // Buffer per la lettura di ogni parola
14     char buffer[MAX_WORD_LENGTH];
15
16     printf("Inserisci il numero di parole (massimo %d): ", N);
17     scanf("%d", &numParole);
18
19     // Verifica che il numero di parole sia valido
20     if (numParole <= 0 || numParole > N) {
21         printf("Numero non valido. Assicurati di inserire un numero compreso tra 1 e %d.\n", N);
22         return 1; // Termina il programma con un codice di errore
23     }
24
25     // Pulisce il buffer di input
26     while (getchar() != '\n');
27
28     // Loop per l'inserimento delle parole
29     for (i = 0; i < numParole; ++i) {
30         // Chiede all'utente di inserire una parola
31         printf("Inserisci la parola %d: ", i + 1);
32         scanf("%s", buffer);
33         // Alloca dinamicamente una nuova stringa della dimensione esatta della parola
34         // INSERITE QUI LA VOSTRA ISTRUZIONE
35         // Copia la parola nel nuovo spazio allocato
36         strcpy(parole[i], buffer);
37     }
38
39     // Stampa la frase ottenuta
40     printf("\nFrase ottenuta: ");
41     for (i = 0; i < numParole; ++i) {
42         printf("%s ", parole[i]);
43     }
44     printf("\n");
45
46     // Libera la memoria allocata dinamicamente
47     for (i = 0; i < numParole; ++i) {
48         free(parole[i]);
49     }
50
51     return 0; // Termina il programma con successo
52 }
```

FIGURA 1. Codice parziale della soluzione.